

LAB # 04

Working with SQL Operators and Scalar Functions

Objectives:

After completing this lab, students will be able to:

- Apply arithmetic operators in SQL queries.
- Use logical operators for conditional expressions.
- Understand and apply operator precedence correctly.
- Recognize and use essential SQL keywords such as DISTINCT, ORDER BY, and aliases.
- Understand scalar functions in SQL.
- Use built-in scalar functions for data manipulation.

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical calculations on numeric operands. These operators allow us to carry out common arithmetic operations such as addition, subtraction, multiplication, and division.

The main arithmetic operators are addition (+), subtraction (−), multiplication (*), and division (/).

In addition to performing operations on numbers, the plus (+) and minus (−) operators can also be used in date arithmetic, such as adding days to a date or calculating the difference between two dates.

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

Table 1: Arithmetic Operators in SQL

Syntax:

```
SELECT expression <arithmetic operator> expression
```

```
FROM table_name  
WHERE condition;
```

Example:

Query: Display marks increased by 5 for all students from Student table.

```
SELECT marks + 5 AS updated_marks  
FROM Student;
```

Logical Operators

AND, OR & NOT operator:

The WHERE clause can be combined with AND, OR, and NOT operators.

The AND and OR operators are used to filter records based on more than one condition:

- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.

The NOT operator displays a record if the condition(s) is NOT TRUE.

AND Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

OR Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

BETWEEN operator:

The BETWEEN operator selects values within a given range. The values can be numbers, text, or dates.

The BETWEEN operator is inclusive: begin and end values are included.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

LIKE operator:

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the LIKE operator:

“%” □ The percent sign represents zero, one, or multiple characters

“_” □ The underscore represents a single character

The percent sign and the underscore can also be used in combinations!

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE column_name LIKE pattern;
```

Here are some examples showing different LIKE operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a_'	Finds any values that 2 nd last char is "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Example:

Query: Display all records of employees whose first name starts with “E”.

```
SELECT *  
FROM employees
```

```
WHERE FIRST_NAME LIKE 'E%';
```

IN Operator:

The IN operator allows you to specify multiple values in a WHERE clause.

The IN operator is a shorthand for multiple OR conditions.

Syntax:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE column_name IN (value1, value2, value3, ...);
```

Example:

Query: Display all employees whose department ID is 10, 20, or 30.

```
SELECT *  
FROM employees  
WHERE department_id IN (10, 20, 30);
```

Operator Precedence

Operator precedence describes the order in which operations are performed when an expression is evaluated.

Operations with a higher precedence are performed before those with a lower precedence.

Parentheses have the highest precedence and OR has the lowest.

Precedence	Operator Type	Examples
1	Parentheses	()

Lab # 04

Working with SQL Operators and Scalar Functions

2	Arithmetic	*, /, +, -
3	Comparison	=, !=, <>, <, >, <=, >=, BETWEEN, IN, LIKE, IS NULL
4	Logical NOT	NOT
5	Logical AND	AND
6	Logical OR	OR

Table 2: Operator Precedence Table

DISTINCT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values.

Inside a table, a column often contains many duplicate values; and sometimes you only want to list the different (distinct) values.

Syntax:

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

Aliases

SQL aliases are used to give a table, or a column in a table, a temporary name. Aliases are often used to make column names more readable.

An alias only exists for the duration of the query.

Alias Column Syntax:

```
SELECT column_name AS alias_name
FROM table_name;
```

Alias Table Syntax:

```
SELECT column_name(s)
FROM table_name AS alias_name;
```

ORDER BY Keyword

The ORDER BY keyword is used to sort the result-set in ascending or descending order.

The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword.

Syntax:

```
SELECT column_name(s)
FROM table_name
ORDER BY column1, column2, ... ASC|DESC
```

SELECT Top Clause

The SELECT TOP clause is used to specify the number of records to return.

It is useful on large tables with thousands of records because returning too many records can impact performance.

Note: Not all database systems support the SELECT TOP clause.

- MySQL uses the **LIMIT** clause to select a limited number of records.
- Oracle uses **ROWNUM** or **FETCH FIRST** to achieve the same result.

Oracle Syntax Using ROWNUM

```
SELECT *
FROM table_name
WHERE ROWNUM <= number;
```

Example:

Query: Return the first 5 rows:

```
SELECT *
FROM employees
WHERE ROWNUM <= 5;
```

Oracle Syntax Using FETCH FIRST (Oracle 12c and later)

```
SELECT *
FROM table_name
ORDER BY column_name
FETCH FIRST number ROWS ONLY;
```

Example:

Query: Return the first 5 rows ordered by employee_id:

```
SELECT *
FROM employees
ORDER BY employee_id
FETCH FIRST 5 ROWS ONLY;
```

Offset in Oracle

In Oracle, the **OFFSET** keyword is used to skip a number of rows:

```
SELECT *
FROM table_name
ORDER BY column_name
OFFSET offset_number ROWS
```

Lab # 04

Working with SQL Operators and Scalar Functions

`FETCH NEXT` number `ROWS ONLY`;

Example:

Query: Skip first 5 rows, return next 5 rows:

```
SELECT *  
FROM employees  
ORDER BY employee_id  
OFFSET 5 ROWS  
FETCH NEXT 5 ROWS ONLY;
```

Functions with SELECT Statement

There are some functions that can be used in select statement.

Syntax:

```
SELECT function_name()  
FROM table_name;
```

Functions are: MIN, MAX, AVG, SUM, COUNT, UPPER, LOWER, LENGTH, etc.

SQL Functions

A function is a predefined formula which takes one or more arguments as input then process the arguments and returns an output.

SQL Functions:

There are two types of SQL functions, aggregate functions, and scalar (non-aggregate) functions.

1. Scalar (Non-Aggregate) / Single Row Functions

Non-aggregate functions operate on each record independently.

OR

These functions operate of single rows only and return one result per row and can be used in SELECT, WHERE and ORDER BY clause.

2. Aggregate / Multiple Rows Functions

Aggregate functions operate on many records and produce a summary, works with GROUP BY.

OR

These functions manipulate group of rows to give one and only one result per group of rows.

There are so many built-in functions in SQL to do various calculations on data.

Types of Scalar Functions

1. **Character Functions:** Accept character input and can return number or character value.
2. **Numeric Functions:** Accept numeric input and returns numeric values.

3. **Date Functions:** Operate on values of date data type. It always return a date value except the MONTHS_BETWEEN function, which returns a number.
4. **Conversion Functions:** Converts a value from one data type to another.
5. **General Functions:** Usually contains NULL handling functions.

1. Character Functions

LOWER() function

LOWER() function is used to convert all characters of a string to lower case.

Syntax:

```
SELECT LOWER(string)
```

UPPER() function

UPPER() function is used to convert all characters of a string to uppercase.

Syntax:

```
SELECT UPPER(string)
```

TRIM() function

The TRIM() function removes leading and trailing spaces from a string.

Syntax:

```
TRIM(string)
```

CONCAT() function

The CONCAT() function adds two or more expressions together.

Syntax:

```
CONCAT(expression1, expression2, expression3,...)
```

LENGTH Function

The LENGTH() function returns the length of a string.

Syntax:

```
LENGTH(string)
```

REPLACE() Function

The REPLACE() function replaces all occurrences of a substring within a string, with a new substring.

Note: This function performs a case-sensitive replacement.

Syntax:

```
REPLACE(string, from_string, new_string)
```

SUBSTR() Function

Lab # 04

Working with SQL Operators and Scalar Functions

These functions extract a substring from a string (starting at any position).

Syntax:

```
SELECT SUBSTR(string, start, length)
```

Parameter	Description
string	Required. The string to extract from
start	Required. The start position. Can be both a positive or negative number. If it is a positive number, this function extracts from the beginning of the string. If it is a negative number, this function extracts from the end of the string
length	Required. The number of characters to extract

INSTR() Function

The INSTR() function returns the position of the first occurrence of a string in another string. This function performs a case-insensitive search.

Syntax:

INSTR(string1, string2)	
Parameter	Description
string1	Required. The string that will be searched
string2	Required. The string to search for in string1. If string2 is not found, this function returns 0

2. Number Functions

MOD() Function

The MOD() function returns the remainder of a number divided by another number.

Syntax:

```
MOD(x, y)
```

ROUND() Function

The ROUND() function rounds a number to a specified number of decimal places.

Syntax:

```
ROUND(number, decimals)
```

TRUNC() Function

The TRUNC() function truncates a number to the specified number of decimal places.

Syntax:

```
TRUNC(number, decimals)
```

CEIL() Function

The CEIL() function returns the smallest integer value that is bigger than or equal to a number.

Syntax:

```
CEIL(number)
```

3. Date Functions

Date Difference

In Oracle, subtracting two dates returns the number of days between them.

Syntax:

```
date2 - date1
```

Example:

```
SELECT TO_DATE('2026-02-10','YYYY-MM-DD') - TO_DATE('2026-02-05','YYYY-MM-DD') AS days_diff  
FROM dual;
```

Add Date / Time

Add days or months to a date using arithmetic or ADD_MONTHS().

Add days: date + number_of_days

Add months: ADD_MONTHS(date, number_of_months)

Example:

```
SELECT SYSDATE + 10 AS future_date,  
ADD_MONTHS(SYSDATE, 2) AS future_month  
FROM dual;
```

CURRENT_DATE() Function

The CURRENT_DATE function returns the current date in the session time zone.

Syntax:

```
CURRENT_DATE
```

Example:

```
SELECT CURRENT_DATE FROM dual;
```

CURRENT_TIME() Function

The CURRENT_TIME function returns the current time.

Syntax:

```
CURRENT_TIME
```

Example:

```
SELECT CURRENT_TIME FROM dual;
```

CURRENT_TIMESTAMP() Function

The CURRENT_TIMESTAMP function returns the current date and time.

Syntax:

CURRENT_TIMESTAMP

Example:

```
SELECT CURRENT_TIMESTAMP FROM dual;
```

TRUNC() Function

The TRUNC() function extracts the date part from a datetime value (removes the time portion).

Syntax:

TRUNC(date)

Example:

```
SELECT TRUNC(SYSDATE) AS today_date FROM dual;
```

Other Date Functions

- EXTRACT(MINUTE FROM date) — Returns the minute part (0–59)
- EXTRACT(HOUR FROM date) — Returns the hour part (0–23)
- EXTRACT(DAY FROM date) — Returns the day of the month (1–31)
- EXTRACT(MONTH FROM date) — Returns the month (1–12)
- EXTRACT(YEAR FROM date) — Returns the year (1000–9999)

Example:

```
SELECT EXTRACT(DAY FROM SYSDATE) AS day,  
EXTRACT(MONTH FROM SYSDATE) AS month,  
EXTRACT(YEAR FROM SYSDATE) AS year  
FROM dual;
```

4. Conversion Functions

TO_CHAR() Function

The TO_CHAR() function formats a DATE value into a specified string format.

Syntax:

TO_CHAR(date, format)

Example:

```
TO_CHAR(DATE '2017-06-15', 'Month DD YYYY')
```

TO_CHAR() Function

The TO_CHAR() function formats a NUMBER into a specified numeric format and returns it as a string.

Syntax:

TO_CHAR(number, format)

Example:

```
TO_CHAR(1234567.89, '9,999,999.99')
```

CAST()

Converts a value from one datatype to another using SQL standard syntax.

Syntax:

CAST(value AS datatype)

Example:

```
CAST('2500' AS INTEGER)
```

5. General Functions

NULLIF() Function

The NULLIF() function compares two expressions and returns NULL if they are equal; otherwise, it returns the first expression.

Syntax:

NULLIF(expr1, expr2)

Example:

```
SELECT NULLIF(10, 10) FROM dual; -- returns NULL  
SELECT NULLIF(10, 5) FROM dual; -- returns 10
```

COALESCE() Function

The COALESCE() function returns the first non-NULL value from a list of expressions.

Syntax:

COALESCE(val1, val2, ..., val_n)

Example:

```
SELECT COALESCE(NULL, NULL, 25, 50) FROM dual; -- returns 25
```

NVL() Function

The NVL() function returns a specified value if the expression is NULL; otherwise, it returns the expression itself.

Syntax:

NVL(expression, alt_value)

Example:

```
SELECT NVL(NULL, 100) FROM dual; -- returns 100  
SELECT NVL(50, 100) FROM dual; -- returns 50
```

Lab tasks

1. Display all employees whose salary is greater than 9000.
2. Show employees who were hired before 01-JAN-2010.
3. Display employees working in department 30 or 70.

4. Show employees whose salary is between 4000 and 10000.
5. Display employees whose commission_pct is NOT NULL.
6. Show employees whose first_name starts with 'M'.
7. Display employees whose first_name ends with 'y'.
8. Show employees whose first_name contains 'an'.
9. Display employees whose first_name has exactly 6 characters.
10. Show employees whose first_name starts with 'J' and ends with 'n'.
11. Display employees whose salary is greater than 12000 and department_id is 80.
12. Show employees whose salary is greater than 7000 or department_id is 40.
13. Display employees whose salary is NOT greater than 6000.
14. Show employees who are in department 30 or 90 and earn more than 5000.
15. Display employees whose salary is less than 20000 and not in department 50.
16. Show how many days each employee has worked till today.
17. Display each employee's contract end date assuming 6 months after hire_date.
18. Show today's date without time.
19. Extract the year and month from each employee's hire_date.
20. Display the difference in days between '2026-03-01' and '2026-02-20'.
21. Write a query that returns NULL if two numbers 80 and 80 are equal.
22. Return the first non-null value from: NULL, 500, 700.
23. Display employee name and show 0 if commission_pct is NULL.
24. Return 999 if NULL is passed, otherwise return the value itself.
25. Display employee first_name and salary formatted with a dollar sign and commas